Week 7 슬라이드 자습 가이드

Virtual Memory (Application View): VAS, brk()/mmap(), ptmalloc · 최신 강의자료 v10 (총 61쪽) 기준

이 자료를 사용하는 방법

영문 강의 슬라이드를 보면서 읽는 한국어 학습 가이드입니다. 원본 예시의 형식인 슬라이드별 내용 요약 / 기억할 점을 유지하되 최신 v10의 페이지에 맞추었습니다. 슬라이드 1·3·18·37·61은 표지 또는 구간 안내여서 행별 해설에서 제외했습니다.

이번 주 학습 목표 — ① ELF → PT_LOAD → VMA를 설명한다. ② malloc()과 brk()/mmap()의 관계 및 VmSize/RSS/PSS 차이를 설명한다. ③ ptmalloc의 chunk 재사용과 free() 이후 RSS가 유지되는 이유를 설명한다.

추천 학습 순서 — 슬라이드의 Key Points → 중앙 예제/도식 → Key Takeaway → 아래 요약표 순으로 확인합니다. 교시별 Checkpoint(17·36·59쪽)는 먼저 스스로 답한 뒤 이 문서 뒷부분의 정답과 비교합니다.

실습 주의 — 주소와 출력은 Ubuntu 24.04 / glibc 2.39 / x86-64를 예시로 하며 시스템과 ASLR 상태에 따라 달라질 수 있습니다. /dev/mem, MMIO 예시는 실제 하드웨어 주소가 아닌 개념 설명이며 임의의 환경에서 그대로 실행하면 안 됩니다.

쪽	슬라이드	내용 요약	기억할 점
2	Roadmap of Part 3	7주차는 애플리케이션에서 보이는 가상 주소 공간, 8주차는 주소 변환과 page fault, 9주차는 memory pressure/reclaim, 10주차는 OOM/LMK를 다룹니다.	이번 주는 user-space 관점이 중심입니다. page table의 상세 동작은 8주차로 넘깁니다.

1부. 프로세스의 메모리는 어디에 있을까요? (4-17쪽)

이 블록은 ELF에 기록된 매핑 정보가 실행 중 VMA로 이어지는 과정과, 커널이 VMA를 이용해 주소·권한·backing을 관리하는 방식을 다룹니다.

쪽	슬라이드	내용 요약	기억할 점
4	Recap: Virtual Address Space (VAS)	&global, &local, malloc 결과의 포인터는 모두 가상 주소입니다. text/data/heap/공유 라이브러리/stack은 개념적인 메모리 영역입니다. MMU는 page table을 이용해 주소를 변환하며 유효한 변환이 없으면 page fault가 발생할 수 있습니다.	가상 주소 ≠ 물리 주소. page fault의 원인과 처리는 8주차에서 자세히 다룹니다.
5	ELF Overview: From Objects to One ELF	main.o, util.o, crt*.o의 .text/.rodata/.data/.bss를 링커가 통합하여 ELF를 생성합니다. libc.so 같은 공유 라이브러리는 보통 코드가 복사되는 것이 아니라 동적 의존성이 기록됩니다.	Section은 링크 관점이며, ELF에는 Header·PHT·SHT 등이 있습니다.
6	ELF Overview: Sections and Segments	SHT는 section 정보를, PHT는 loader가 참조하는 segment 정보를 제공합니다. 비슷한 permission을 가지는 section들이 PT_LOAD로 묶입니다. 예제는 R/RE/R/RW LOAD를 구분합니다.	Section은 linker, segment는 loader 관점. 런타임 로딩에는 PHT가 핵심입니다.
7	PT_LOAD Details: Sizes and Addresses	PT_LOAD의 Offset·VirtAddr·FileSiz·MemSiz·Flags를 확인합니다. MemSiz가 FileSiz보다 크면 초과 영역은 0으로 초기화됩니다. PIE에서 runtime address는 ELF VA에 load bias를 더해 계산합니다.	MemSiz > FileSiz와 .bss를 연결합니다. 한 segment가 최종적으로 여러 VMA가 될 수 있습니다.
8	Checking PT_LOAD with readelf	readelf -lW로 LOAD의 offset/VA/filesz/memsz를, -SW로 section 주소와 파일 오프셋을 봅니다. LOAD4 예제에서 0x270-0x268=8B가 파일 외 0 초기화 영역입니다. g_init 주소도 section offset으로 검산합니다.	readelf 숫자로 section VA = segment VA + (section offset - segment offset)를 확인합니다.

쪽	슬라이드	내용 요약	기억할 점
9	Mapping PT_LOAD to Process VAS	execve 시 커널은 PHT의 PT_LOAD를 참고해 매핑을 설정합니다. 예제에서 LOAD1~3은 초기 VMA, LOAD4는 RELRO 변화 후 권한이 다른 VMA로 보입니다. heap과 stack은 해당 ELF의 PT_LOAD가 아닙니다.	PT_LOAD와 VMA는 1:1 불변 관계가 아닙니다. 매핑은 demand paging으로 실제 페이지를 나중에 채웁니다.
10	Why One Segment Becomes Several VMAs	LOAD4는 처음 writable하게 매핑된 후 동적 링커가 relocation을 수행하고, RELRO 영역을 mprotect로 read-only 처리합니다. .bss의 anonymous backing도 VMA 경계를 만들 수 있습니다.	Permission 또는 backing이 바뀌면 한 segment가 여러 VMA로 나타날 수 있습니다.
11	Textbook Layout vs. Real VMAs	text/data/heap/stack의 단순 그림과 실제 /proc/<pid>/maps의 많은 VMA를 비교합니다. 공유 라이브러리·anonymous·권한별 분할로 개념적 영역보다 실제 VMA 수가 많습니다.	커널의 주소 범위 관리 단위는 VMA. 실제 시스템에서는 maps를 직접 확인합니다.
12	Why Do Addresses Change? PIE and ASLR	PIE 실행 파일은 load bias가 실행마다 달라질 수 있습니다. ASLR은 실행 파일·heap·shared lib·stack 등을 무작위화하므로 동일 프로그램의 주소가 매번 다르게 출력됩니다.	주소 변화는 정상입니다. setarch -R은 통제된 실험에서만 사용합니다.
13	Observing the Address Space	/proc/<pid>/maps에는 VMA당 한 행이 나오고 주소 오름차순으로 표시됩니다. 같은 실행 파일/라이브러리라도 권한별로 여러 행이 존재합니다. [vvar]/[vdso]도 함께 관찰합니다.	maps의 한 줄 = 하나의 VMA. 파일 이름 하나가 여러 줄로 나올 수 있습니다.
14	Reading One Mapping	각 행에서 시작-끝 주소, rwx와 p/s permission, file offset, dev, inode, pathname을 읽습니다. end address는 포함되지 않으며 anonymous 매핑은 일반 파일 경로가 없습니다.	주소 범위의 크기와 r-xp/rw-p 의미를 실제 예제로 해석해 봅니다.
15	How the Kernel Manages VMAs	프로세스의 mm_struct는 VMA들을 주소 순서로 찾아야 합니다. Linux 6.1+는 Maple Tree, 이전 Linux는 rb-tree와 linked list 기반 구조를 사용했습니다. 주요 vm_area_struct 필드와 maps 열을 연결합니다.	Maple Tree는 VMA 인덱스이지 page table이 아닙니다. 노드가 주소 범위를 다룹니다.
16	How Can VMA Be Used?	Page fault가 나면 커널은 해당 주소가 속한 VMA를 찾고 접근 permission과 backing을 확인합니다. VMA가 없거나 보호를 위반하면 SIGSEGV가 발생할 수 있고, 유효하면 page fault를 처리할 수 있습니다.	VMA는 유효 주소·접근 권한·backing을 판단하는 근거입니다. 상세 fault 경로는 8주차.
17	Checkpoint: Process VAS	ELF section/segment, segment→VMA 분할, ASLR, maps 한 줄, VMA 외 접근, Linux 6.1+ 인덱스 구조를 확인하는 6문항입니다.	정답을 보기 전에 ELF에서 실행 중 VMA가 만들어지는 과정을 말로 설명해 봅니다.

2부. brk()와 mmap()은 주소 공간을 어떻게 바꿀까요? (19-36쪽)

핵심 질문은 malloc이 커널을 언제 호출하는지, VMA가 커질 때 실제 물리 메모리 사용량도 함께 늘어나는지입니다.

쪽	슬라이드	내용 요약	기억할 점
19	From Static Layout to Dynamic Growth	실행 직후 주소 공간에는 stack·lib·text/data 등이 존재합니다. malloc은 기존 chunk를 재사용하거나, brk로 heap을 키우거나, mmap으로 별도 매핑을 만듭니다.	reuse / brk / mmap의 세 갈래를 /proc/<pid>/maps 전후 비교로 관찰합니다.

쪽	슬라이드	내용 요약	기억할 점
20	How Does the Kernel Find Address Space?	brk는 기존 program break의 확장 가능 여부를 확인합니다. mmap은 VMA 사이에서 요청 크기를 담을 gap을 찾아야 하며, Maple Tree는 subtree의 gap 요약 정보를 이용할 수 있습니다.	brk는 heap 끝 확장, mmap은 적절한 빈 구간 탐색. 실제 배치는 정책·ASLR에도 좌우됩니다.
21	Is malloc() a System Call?	malloc은 glibc의 사용자 공간 함수입니다. 기존 allocator 공간에 요청이 맞으면 kernel 진입 없이 반환하며, 공간이 필요할 때만 brk/mmap 시스템 콜을 유발합니다.	malloc 자체는 syscall이 아닙니다.
22	Observing malloc() with strace	strace에는 malloc/free 함수 자체가 나타나지 않고 brk/mmap/munmap 같은 시스템 콜이 기록됩니다. 첫 작은 malloc이 heap을 크게 확장하고 뒤 요청은 재사용되며 큰 요청은 별도 mmap을 사용합니다.	malloc 횟수 ≠ syscall 횟수. free가 syscall 없이 끝날 수 있습니다.
23	Why Doesn't malloc() Call the Kernel Every Time?	매 할당을 syscall로 처리하면 overhead가 큽니다. glibc가 큰 영역을 확보해 작은 chunk로 나누고 반복 재사용하므로 malloc(64)을 1만 번 호출해도 시스템 콜 수는 훨씬 적을 수 있습니다.	Allocator는 system-call 빈도와 latency를 줄이기 위해 큰 영역을 관리합니다.
24	What is brk()?	program break를 옮겨 heap을 키웁니다. malloc(1024) 요청 한 번에 0x21000(135,168B, 약 132KiB)의 heap 공간을 확보하는 예를 보여줍니다. 물리 페이지는 실제 사용 시 fault로 생길 수 있습니다.	가상 heap 증가와 resident page 증가를 구별해야 합니다.
25	Observing brk() in /proc/<pid>/maps	break만 설정된 초기 상태와 첫 malloc으로 [heap] VMA가 만들어지는 모습, 두 번째 작은 malloc에서 VMA가 그대로인 모습을 비교합니다.	첫 malloc은 큰 공간을 받고, 두 번째는 내부에서 재사용할 수 있습니다.
26	What is mmap()?	mmap은 virtual address range에 매핑을 설정합니다. Anonymous(MAP_ANONYMOUS), file-backed(fd), device-backed(driver callback)의 backing 사례를 비교합니다.	mmap으로 만든 매핑이 곧 모든 물리 페이지의 즉시 할당을 뜻하지는 않습니다.
27	Anonymous Mapping Example	1MiB anonymous private mmap 뒤 memset으로 접근하여 첫 write fault에서 zero-filled physical pages가 연결되는 과정을 보여줍니다. munmap은 매핑을 제거합니다.	먼저 VMA, 나중에 page residency: on-demand allocation.
28	File-Backed Mapping Example	파일을 열어 MAP_SHARED로 mmap하면 load/store로 내용을 다룰 수 있습니다. 최초 접근 시 page cache를 통한 fault가 가능하고 쓰기 시 dirty page가 만들어져 나중에 write-back됩니다.	파일 동기화가 필요하면 msync(MS_SYNC)를 사용합니다. mmap도 page cache 관리를 피하는 것은 아닙니다.
29	Device-Backed Mapping Example	개념상의 /dev/mydev 드라이버가 mmap callback에서 MMIO 레지스터 또는 DMA buffer를 프로세스의 VMA로 매핑하는 사례입니다.	일반 메모리와 MMIO는 다릅니다. 매핑 속성과 접근 방법은 드라이버가 정의합니다.
30	Device-Backed Mapping Without Device Driver Example	/dev/mem은 물리 주소를 offset으로 지정해 특수 범위를 매핑하는 데 사용될 수 있습니다. root 권한, CONFIG_STRICT_DEVMEM, 실제 장치 주소가 필요합니다.	설명용 물리 주소를 실제 장비에서 실행하지 않습니다. 일반 애플리케이션 패턴이 아닙니다.

쪽	슬라이드	내용 요약	기억할 점
31	Observing mmap() with /proc/<pid>/maps	file-backed는 pathname/inode/dev가, anonymous는 일반 파일 이름과 inode가 없는 모습이 나타납니다. 예제의 /dev/zero는 특수 device file이지 실제 MMIO 매핑 사례가 아닙니다.	perms·offset·dev·inode·path를 함께 읽어야 매핑 성격을 판단할 수 있습니다.
32	malloc() Uses brk() and mmap(): mmap Threshold	작은 요청은 heap에서, 큰 요청은 별도 mmap에서 처리되는 경향이 있습니다. 다만 요청 크기, free 공간, allocator 상태와 glibc tunable이 모두 관여합니다. Non-main arena heap도 mmap으로 생성될 수 있지만 M=1 mmap chunk와는 다릅니다.	128KiB는 대표적 초기 threshold이며 커널 고정 규칙이 아닌 glibc 정책입니다.
33	Observing mmap Threshold is Dynamic	100KB allocation과 첫 200KB 요청 후 mmap/free가 threshold를 올리는 실험입니다. 이후 같은 200KB 요청이 heap/brk에서 처리될 수 있습니다. GLIBC_TUNABLES 또는 mallopt로 정책을 고정할 수 있습니다.	Threshold는 실행 중 바뀔 수 있고 고정하면 적응이 꺼집니다.
34	Virtual Size vs Resident Size	100MiB malloc 직후 가상 주소 범위(VmSize)는 크게 늘지만 RSS는 거의 그대로일 수 있습니다. memset으로 전체 페이지를 쓰면 fault를 거쳐 RSS가 약 100MiB 증가합니다.	VmSize = 주소 공간, RSS = 현재 resident page. 증가 시점이 다릅니다.
35	Observing Memory Usage with smaps or smaps_rollup	smaps는 VMA별, smaps_rollup은 프로세스 합산 통계를 제공합니다. RSS는 공유 페이지를 각 프로세스에서 전부 세지만 PSS는 공유 비율대로 나눠 계상합니다.	공유가 있는 여러 프로세스의 메모리 합산·비교에는 PSS가 유용합니다.
36	Checkpoint: Dynamic VAS	malloc의 시스템 콜 여부, brk/mmap 변경점, mmap 직후 page, mmap 선택 정책, malloc 100MiB의 VmSize/RSS, PSS 이유를 확인합니다.	각 답을 단순 암기하기보다 VMA 변화와 physical memory 변화로 풀어봅니다.

3부. ptmalloc은 메모리를 어떻게 관리할까요? (38-59쪽)

이 부의 핵심은 allocator가 chunk를 빠르게 재사용하는 대신 fragmentation·memory retention을 만들 수 있으며, free 이후 RSS는 allocator와 kernel의 메모리 회수 경로에 따라 달라진다는 점입니다.

쪽	슬라이드	내용 요약	기억할 점
38	Memory Allocators in General	malloc은 표준 API이며 내부 구현은 glibc ptmalloc, jemalloc, TCMalloc, mimalloc 등으로 달라집니다. latency, fragmentation, footprint, scalability 사이에서 서로 다른 설계를 선택합니다.	이번 강의 대상은 glibc ptmalloc이며 장기 실행 임베디드 서비스의 RSS에 특히 중요합니다.
39	Allocation Unit: What is a Chunk?	Allocator는 요청한 byte가 아니라 header와 payload를 포함하는 chunk 단위로 관리합니다. 반환 포인터는 header가 아닌 payload 시작을 가리킵니다. 정렬과 최소 chunk 크기가 있습니다.	chunk 크기는 요청량보다 클 수 있습니다.
40	Fragmentation in Memory Allocators	Internal fragmentation은 chunk 안의 metadata/padding/미사용 공간입니다. External fragmentation은 free chunk가 불연속적으로 나뉘어 큰 요청을 곧바로 충족하지 못하는 상황입니다.	총 free memory가 충분해도 연속한 큰 free chunk는 없을 수 있습니다.
41	Free Chunk Management: Free List	Allocator는 free chunk를 singly/doubly linked list 또는 크기별 bin으로 관리합니다. coalescing과 splitting으로 chunk를 합치고 나눕니다.	list 설계와 merge/split 정책이 검색 비용·fragmentation의 균형을 정합니다.

쪽	슬라이드	내용 요약	기억할 점
42	Two Performance Metrics: Latency vs Memory Utilization	한 번의 malloc/free 지연시간(평균·tail), throughput과 유효 payload 비율 U=live payload/managed memory를 비교합니다. 빠른 재사용은 높은 footprint와 맞바꿀 수 있습니다.	Speed vs. Memory utilization이 allocator 설계의 핵심 trade-off입니다.
43	Overview of malloc() Chunk (64-bit ptmalloc)	prev_size·size 필드와 P/M/A 플래그, free chunk 내부 fd/bk를 봅니다. P=앞 chunk 사용 여부, M=독립 mmap chunk 여부, A=non-main arena 여부입니다. prev_size는 앞 chunk가 free일 때 의미가 있습니다.	P bit는 현재가 아닌 이전 chunk를 설명합니다. 16B 형식과 실효 8B overhead는 양립합니다.
44	mmap Chunk	독립적인 mmap allocation의 chunk는 M=1이며 자체 mapping에 존재하고 arena bin에 들어가지 않습니다. free 시 일반적으로 전체 mapping을 munmap합니다. 다른 arena의 mmap-backed heap에서 나온 M=0 chunk와 구별합니다.	mmap syscall 이용 ≠ 모든 chunk의 M=1. 직접 mmap한 사용자 영역도 glibc chunk가 아닙니다.
45	Minimum Chunk Size (64-bit ptmalloc)	전형적인 x86-64 glibc에서 16B alignment·최소 32B를 사용합니다. chunk=max(32,align16(n+8)); malloc(1)/malloc(24)는 32B, malloc(25)는 48B, malloc(62)는 80B입니다.	malloc(24)→32B, malloc(25)→48B. 한 바이트 증가가 size class를 바꿀 수 있습니다.
46	Overall Structure of Free Chunks in ptmalloc	tcache는 스레드 전용, arena는 스레드가 공유할 수 있는 관리 영역입니다. Arena의 fastbin·unsorted·smallbin·largebin과 top chunk를 구분합니다. 표의 개수·경계는 예시 glibc 설정 기준입니다.	tcache hit는 arena lock 없이 빠르고, miss 때 공유 arena 경로로 갈 수 있습니다.
47	Details of Free Chunk Structure in ptmalloc	tcache/fastbin은 LIFO 연결 리스트, smallbin은 exact-size FIFO, largebin은 size-ordered range, unsorted는 임시 보관·분류 지연, top은 heap 끝의 남은 공간입니다.	모든 free chunk를 하나의 목록에 넣지 않는 이유는 성능·단편화 trade-off 때문입니다.
48	tcache Hit vs Miss	맞는 tcache size class가 비어 있지 않으면 head를 꺼내 O(1)로 반환합니다. miss일 때만 arena 쪽 검색을 시도하며, fastbin/smallbin 경로에서는 남는 동일 class chunk를 cache로 채울 수 있습니다.	tcache는 다른 크기 class를 검색하거나 큰 chunk를 나누지 않습니다.
49	Top Chunk	Top chunk는 arena의 heap 끝에 존재하는 free 영역이며 bin이 아닙니다. 다른 경로에서 찾지 못한 요청은 top에서 분할할 수 있고, 부족하면 heap이 확장됩니다. 인접 free가 병합되고 조건에 맞으면 trim할 수 있습니다.	Top은 heap의 확장·축소와 연결되는 예비 공간입니다. threshold와 pad는 정책값입니다.
50	malloc() Path for Small Chunks	개념적 경로: tcache → fastbin/smallbin → unsorted scan → 큰 bin에서 split → top/heap growth. 실제 _int_malloc의 제어 흐름은 크기·상태·버전에 따라 분기가 다릅니다.	그림은 역할 설명용이며 고정된 실제 실행 순서라고 해석하면 안 됩니다.
51	Example: malloc() with Small Chunks	48B fastbin hit 후 남은 48B chunk가 tcache refill, 다음 요청은 tcache hit가 되는 예시입니다. 이어 112B smallbin 사용, unsorted의 304B를 분류하고 208B exact fit 선택, 2,000B free를 512B와 remainder로 split하는 개념 예를 보여줍니다.	Exact fit ≠ 단지 크기가 충분함. 이 예제는 교육용 allocator 상태이지 실제 실행 trace는 아닙니다.
52	malloc() Path for Large Chunks	1,024B 이상 요청을 개념적으로 설명합니다. fastbin consolidation, unsorted 검사, largebin best fit, 더 큰 bin/top, 메모리 확보를 차례로 보여주되 실제 분기·size class는 설정에 의존합니다.	큰 chunk도 단순히 무조건 mmap을 사용하는 것은 아닙니다.

쪽	슬라이드	내용 요약	기억할 점
53	Example: malloc() with Large Chunks	1500B 요청은 chunk 1520B로 정렬되고, 인접하다고 가정한 fastbin free chunk가 병합됩니다. 더 작은 충분한 largebin 항목을 선택·split합니다. 3500B 요청은 top, 매우 큰 요청은 mmap 예를 보여줍니다.	병합은 주소상 인접할 때만 가능. bin에 함께 있다고 합쳐지는 것은 아닙니다.
54	What Happens When free() is Called?	M=1이면 일반적으로 munmap, 작은 M=0 chunk는 tcache/fastbin에 보관, 그 외는 이웃과 병합하거나 top에 합쳐질 수 있습니다. 실제 경로는 크기, 인접 chunk, cache/arena 상태에 따라 달라집니다.	free() = 무조건 OS 반환 아님. tcache/fastbin은 즉시 병합하지 않을 수 있습니다.
55	Does free() Return Physical Memory?	Heap chunk를 free하면 보통 allocator 내부로 돌아가 VMA/RSS가 그대로일 수 있습니다. 독립 mmap chunk는 munmap으로 해당 VMA가 제거되고 resident page가 회수되어 RSS가 감소할 수 있습니다.	해제한 virtual memory와 resident physical page는 다른 개념입니다.
56	Observing RSS After free()	같은 1만 개의 4KiB allocation을 모두 free하는 A와 하나 건너 free하는 B를 비교합니다. A는 top 병합/trim으로 RSS가 떨어질 수 있으나, B는 사용 중 chunk가 사이에 있어 연속 free 공간이 만들어지지 않습니다.	같은 free 총량이라도 해제 패턴에 따라 RSS가 크게 다릅니다.
57	What is malloc_trim()?	malloc_trim은 allocator가 회수 가능한 페이지를 OS로 반환하도록 시도합니다. Heap 끝은 brk 등으로 가상 주소 영역까지 줄 수 있고, 내부의 완전히 빈 페이지는 madvise로 RSS만 줄 수 있습니다.	VmSize가 그대로여도 RSS는 감소할 수 있습니다. 모든 free byte를 반드시 돌려주지는 않습니다.
58	Implications in Memory-Constrained Systems	장기 실행 서비스의 RSS가 작업 이후에도 높은 원인은 allocator retention, partial-page occupancy, multiple arenas, 성능 중심 cache 정책일 수 있습니다. smaps 등으로 관찰 후 trim 여부를 판단합니다.	High RSS ≠ 곧 memory leak. 서비스의 패턴과 물리 메모리 변화를 함께 분석합니다.
59	Checkpoint: Inside the Allocator	chunk/metadata, malloc(33) 크기, 내·외부 단편화, tcache hit/miss, free 후 RSS, malloc_trim 효과를 확인합니다.	할당부터 free/RSS까지 하나의 요청을 따라 설명할 수 있어야 합니다.

Week 7 한 장 요약과 핵심 용어 (60쪽)

프로세스는 고유한 가상 주소 공간을 가지며, glibc malloc은 기존 메모리 재사용이나 brk/mmap으로 공간을 확보합니다. 메모리 해제는 allocator가 시작하는 munmap·trim·madvise와 메모리 압력하의 커널 reclaim·swap·OOM 대응을 구분해서 이해합니다.

쪽	슬라이드	내용 요약	기억할 점
60	Week 7 Summary	① ELF/PT_LOAD와 VMA ② brk/mmap으로 VAS 변경 ③ tcache/bin/top을 통한 glibc malloc 동작 ④ allocator 메모리 반환과 kernel의 reclaim/swap/OOM 대응을 요약합니다.	Memory allocation ≠ physical residency; free ≠ OS release; high RSS ≠ leak.

용어	의미
VAS / VMA	프로세스의 virtual address space / 권한·backing 등이 같은 연속된 가상 주소 범위
ELF / SHT / PHT	실행 파일 형식 / Section Header Table / Program Header Table
PT_LOAD / load bias	로더가 매핑할 segment / ELF VA에 더하는 runtime 기준 주소

용어	의미
RELRO	동적 relocation 후 특정 영역을 읽기 전용으로 보호하여 VMA permission이 분할될 수 있음
brk() / mmap()	program break를 통한 heap 확장 / 새로운 주소 범위 매핑(anonymous·file·device)
VmSize / RSS / PSS	예약된 virtual memory 범위 / resident memory / shared page를 비례 배분한 통계
chunk / header / payload	ptmalloc 내부 관리 단위 / 관리 정보 / malloc이 제공하는 사용 영역
P / M / A flags	PREV_INUSE / IS_MMAPPED / NON_MAIN_ARENA
tcache / arena	thread-local free chunk 캐시 / bin·top을 관리하는 allocator 영역
fastbin / smallbin / largebin	작은 chunk의 빠른 보관 / 동일 크기 free list / 더 큰 크기 범위의 sorted list
unsorted bin / top chunk	새로 free·병합된 chunk의 임시 보관 / heap 끝 free 영역
fragmentation	internal=chunk 내부 낭비; external=사용 가능한 공간의 불연속
malloc_trim()	반환 가능한 heap 공간·빈 물리 페이지를 OS에 반환하도록 시도하는 glibc 함수

Checkpoint 문제와 답 (17, 36, 59쪽)

슬라이드 질문을 보고 먼저 스스로 답한 후 아래의 요약 답안을 확인하십시오.

17쪽. Process VAS

용어	의미
커널은 섹션과 세그먼트 중 무엇을 참고하나요?	실행 시 PT_LOAD segment를 참고합니다. Section은 주로 linker·도구의 개념입니다.
하나의 segment가 여러 VMA인 이유는?	RELRO 등에 따른 permission 변화, file/anonymous backing 차이 등 때문입니다.
maps의 주소가 달라지는 이유는?	PIE 실행 파일의 load bias 및 주소 공간 ASLR 때문입니다.
/proc//maps의 한 줄은?	연속된 주소 범위·권한·file offset/backing 정보를 가진 하나의 VMA입니다.
VMA 밖에 접근하면?	처리 가능한 stack growth 등의 예외를 제외하면 보통 SIGSEGV로 이어집니다.
Linux 6.1+는 VMA를 어떻게 관리하나요?	mm_struct의 Maple Tree로 범위를 인덱싱합니다.

36쪽. Dynamic VAS

용어	의미
malloc()은 syscall인가요?	아니요. glibc 사용자 공간 함수이고 필요할 때만 brk/mmap syscall을 유발합니다.
brk와 mmap은 무엇을 바꾸나요?	brk는 program break/heap 범위를, mmap은 주소 공간의 매핑을 변경합니다.
mmap 성공 시 physical page가 모두 생기나요?	대개 그렇지 않습니다. anonymous/file-backed 매핑의 page는 이후 접근 때 fault로 연결될 수 있습니다.

용어	의미
malloc은 언제 mmap을 선택하나요?	크기 threshold, allocator 상태, free/top 공간 및 tunable 등에 따라 결정됩니다.
malloc(100MiB) 후 만지지 않으면?	VmSize는 약 100MiB 증가할 수 있지만 RSS는 거의 증가하지 않을 수 있습니다.
PSS를 쓰는 이유는?	공유 페이지의 RSS 중복 계상을 완화하고 프로세스 간 사용량을 비교할 수 있습니다.

59쪽. Inside the Allocator

용어	의미
Chunk는 무엇을 포함하고 malloc은 무엇을 반환하나요?	Metadata(header)와 payload를 가진 관리 단위이며 malloc은 payload 포인터를 반환합니다.
malloc(33)의 일반적인 x86-64 chunk 크기는?	33+8을 16B 배수로 올려 48B입니다(최소 32B).
Internal / external fragmentation은?	Chunk 안의 낭비 / free 공간이 흩어진 현상입니다.
tcache hit와 miss의 차이는?	Hit는 thread-local list에서 빠르게 꺼냄; miss는 arena 쪽 경로를 이용할 수 있습니다.
왜 free 후 RSS가 줄지 않나요?	Allocator가 chunk를 보관하고 mapping/physical page를 유지할 수 있기 때문입니다.
malloc_trim은 무엇을 반환하나요?	반환 가능한 끝 공간 또는 내부 빈 physical page를 OS에 반환하도록 시도합니다.